OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SISTEMI OPERATIVI: LEZIONE 4

Contents

1 Introduzione ai Deadlock **6**
1.1 Concetto di Stato di Attesa . . . 6
1.2 Definizione di Deadlock . . . 6

2 Tipi di Risorse e Concetti Base sui Deadlock **6**
2.1 Gestione delle Risorse da Parte dei Thread . . . 7
2.2 Attesa per Risorse Non Disponibili . . . 7
2.3 Deadlock e Dipendenze Circolari . . . 7

3 Deadlock con Più Mutex **8**
3.1 Descrizione del Problema . . . 8
3.2 Soluzione Sicura . . . 8
3.3 Esempio in C con pthreads . . . 8
3.4 Spiegazione Dettagliata dell'Esempio . . . 9

4 Liveness e Fallimenti di Liveness **10**
4.1 Concetto di Liveness . . . 10
4.2 Fallimento di Liveness . . . 10
4.3 Esempio Semplice . . . 10
4.4 Rischi legati alla Concorrenza . . . 10

5 Livelock **11**
5.1 Definizione . . . 11
5.2 Somiglianza e Differenze con il Deadlock . . . 11
5.3 Analogia Intuitiva . . . 11
5.4 Cause del Livelock . . . 11
5.5 Strategia di Prevenzione . . . 11

6 Condizioni Necessarie per il Deadlock **12**
6.1 1. Mutua Esclusione (Mutual Exclusion) . . . 12
6.2 2. Hold and Wait (Trattenimento e Attesa) . . . 12
6.3 3. Nessuna Preemption (No Preemption) . . . 12
6.4 4. Attesa Circolare (Circular Wait) . . . 12

7 Resource-Allocation Graph **13**
7.1 Definizione e Scopo . . . 13
7.2 Tipi di Nodi . . . 13
7.3 Tipi di Archi . . . 13

7.4 Lettura di un Resource-Allocation Graph . . . 13
7.5 Rappresentazione Testuale del Grafo . . . 14

8 Cicli e Rilevazione del Deadlock 15
8.1 Cicli nel Resource-Allocation Graph . . . 15
8.2 Cicli con Risorse Multiple . . . 15
8.3 Esempio di Deadlock con Cicli Multipli . . . 15
8.4 Esempio di Ciclo Senza Deadlock . . . 15
8.5 Osservazioni Generali . . . 16

9 Strategie di Gestione dei Deadlock 16
9.1 1. Ignorare i Deadlock . . . 16
9.2 2. Prevenzione dei Deadlock . . . 16
9.3 3. Evitamento dei Deadlock . . . 17
9.4 4. Rilevazione e Recupero . . . 17
9.5 Osservazioni Generali . . . 17

10 Prevenzione dei Deadlock: Mutua Esclusione 18
10.1 Principio Generale . . . 18
10.2 Mutua Esclusione . . . 18
10.3 Limiti della Prevenzione tramite Mutua Esclusione . . . 18
10.4 Esempio Pratico . . . 18

11 Prevenzione dei Deadlock: Hold-and-Wait 19
11.1 Principio Generale . . . 19
11.2 Strategie per Eliminare Hold-and-Wait . . . 19
11.3 Limiti di queste Strategie . . . 19
11.4 Esempio Pratico . . . 19

12 Prevenzione dei Deadlock: No-Preemption (Preemption delle Risorse) 20
12.1 Principio Generale . . . 20
12.2 Strategia di Preemption delle Risorse . . . 20
12.3 Alternativa: Preemption Selettiva . . . 20
12.4 Esempio Pratico . . . 20

13 Prevenzione dei Deadlock: Eliminazione della Circular Wait 21
13.1 Principio Generale . . . 21
13.2 Strategia: Ordinamento Globale delle Risorse . . . 21
13.3 Motivazione e Spiegazione . . . 21
13.4 Esempio Pratico . . . 21

14 Deadlock Avoidance: Idea di Base 22
14.1 Principio Generale . . . 22
14.2 Informazioni sui Bisogni Futuri . . . 22
14.3 Monitoraggio delle Risorse . . . 22
14.4 Dichiarazione della Massima Necessità . . . 22
14.5 Verifica dello Stato Sicuro . . . 22

15 Safe State e Safe Sequences **23**
15.1 Definizione di Safe State . . . 23
15.2 Notazione delle Sequenze . . . 23
15.3 Condizione per la Sicurezza . . . 23
15.4 Determinazione dello Stato Sicuro . . . 23
15.5 Osservazioni Didattiche . . . 23

16 Safe vs Unsafe States **24**
16.1 Definizione di Safe State . . . 24
16.2 Definizione di Unsafe State . . . 24
16.3 Osservazioni Didattiche . . . 24
16.4 Esempio di Safe State . . . 24

17 Transizione da Safe a Unsafe State **25**
17.1 Scenario di Transizione . . . 25
17.2 Conseguenze della Transizione . . . 25
17.3 Causa del Deadlock . . . 25
17.4 Spiegazione Didattica . . . 25
17.5 Nota Esemplificativa . . . 25

18 Safe State e Deadlock Avoidance **26**
18.1 Principio Generale . . . 26
18.2 Gestione delle Risorse Disponibili . . . 26
18.3 Spiegazione Didattica . . . 26
18.4 Collegamento con Safe State . . . 26

19 Prevenzione dei Deadlock con Claim Edges **27**
19.1 Principio Generale . . . 27
19.2 Conversione in Request Edge . . . 27
19.3 Gestione dei Cicli . . . 27
19.4 Spiegazione Didattica . . . 27
19.5 Esempio Concettuale . . . 27

20 Banker's Algorithm **28**
20.1 Overview . . . 28
20.2 Strutture Dati Richieste . . . 28
20.3 Safety Algorithm (Verifica Stato Sicuro) . . . 28
20.4 Gestione di una Richiesta di Risorsa . . . 29
20.5 Esempio Numerico: Safety Check . . . 29
20.6 Esempio Pratico: Applicazione del Banker's Algorithm . . . 30
20.6.1 Dati Iniziali . . . 30
20.6.2 Tabella delle Risorse . . . 30
20.6.3 Calcolo di Need . . . 30
20.6.4 Risorse Disponibili Iniziali . . . 30
20.6.5 Verifica dello Stato Sicuro (Safety Check) . . . 30
20.6.6 Conclusione . . . 31

21 Rilevamento dei Deadlock e Recovery **31**
21.1 Introduzione 31
21.2 Applicabilità 31
21.3 Considerazioni sul Recupero 31
21.4 Spiegazione Didattica 31

22 Rilevamento dei Deadlock con Wait-For Graphs **32**
22.1 Introduzione 32
22.2 Costruzione del Wait-For Graph 32
22.3 Interpretazione 32
22.4 Esempio Concettuale 32
22.5 Spiegazione Didattica 32
22.6 Nota Operativa 32

23 Rilevamento dei Deadlock con Risorse Multiple **33**
23.1 Introduzione 33
23.2 Strutture Dati Necessarie 33
23.3 Algoritmo di Rilevamento Deadlock (Multiple Resource Instances) 33
23.4 Passi dell'Algoritmo 33
23.5 Spiegazione Didattica 34
23.6 Nota Operativa 34

24 Perché le Risorse Sono Rilasciate Immediatamente nell'Algoritmo di Rilevamento **35**
24.1 Principio Generale 35
24.2 Esempio Numerico 35
24.3 Step 1: Inizializzazione 35
24.4 Step 2: Trova Thread che può completare 35
24.5 Step 3: Continua con altri thread 36
24.6 Step 4: Fine del primo ciclo 36
24.7 Spiegazione Didattica 36
24.8 Conclusione 36

25 Quando Eseguire l'Algoritmo di Rilevamento dei Deadlock **37**
25.1 Introduzione 37
25.2 Strategie di Esecuzione 37
25.3 Spiegazione Didattica 37
25.4 Nota Operativa 38

26 Strategie di Recupero dai Deadlock **38**
26.1 Introduzione 38
26.2 Metodi Principali di Recupero 38
26.3 Obiettivo del Recupero 38
26.4 Spiegazione Didattica 39

27 Recupero dai Deadlock tramite Terminazione dei Processi **39**
27.1 Strategie di Terminazione 39
27.2 Criteri di Selezione dei Processi da Terminare 39
27.3 Ripristino delle Risorse 39

27.4 Spiegazione Didattica 40

28 Criteri per la Scelta dei Processi da Terminare **40**
28.1 Introduzione 40
28.2 Principali Criteri di Selezione 40
28.3 Spiegazione Didattica 41

29 Preemption delle Risorse nel Recupero dai Deadlock **42**
29.1 Introduzione 42
29.2 Procedura di Preemption 42
29.3 Spiegazione Didattica 42

1 Introduzione ai Deadlock

Nei sistemi multiprogrammati, più thread o processi condividono risorse limitate, come CPU, memoria, dispositivi di I/O o semafori. La competizione per l'accesso a tali risorse può generare situazioni complesse, che se non gestite correttamente possono portare a un fenomeno noto come **deadlock**.

Un *thread* è l'unità di esecuzione più piccola di un processo che può essere schedulata dal sistema operativo. Una *risorsa* è qualsiasi entità utilizzabile da un thread, ad esempio una CPU, un file, una porzione di memoria, un dispositivo di I/O.

In un sistema multiprogrammato, più thread possono richiedere contemporaneamente risorse limitate. Quando una risorsa richiesta non è immediatamente disponibile, il thread che la richiede non può proseguire e deve entrare in uno stato di attesa.

1.1 Concetto di Stato di Attesa

Quando un thread richiede una risorsa già occupata da un altro thread, esso non può continuare la sua esecuzione immediatamente. In tal caso, il sistema operativo sposta il thread nello **stato di attesa**.

Lo *stato di attesa* indica che il thread è pronto a proseguire solo quando la risorsa richiesta diventa disponibile. Durante questo periodo, il thread non consuma CPU attivamente, ma resta bloccato fino al rilascio della risorsa.

1.2 Definizione di Deadlock

Un **deadlock** si verifica quando un insieme di processi si trova in una situazione in cui ciascun processo attende un evento che solo un altro processo dello stesso insieme può generare. In altre parole, nessun processo può proseguire perché tutti dipendono reciprocamente da risorse occupate dagli altri.

Spiegazione dettagliata:

- Consideriamo due o più thread che condividono risorse limitate.
- Se ogni thread detiene una risorsa e attende un'altra risorsa già detenuta da un altro thread del gruppo, si genera un ciclo di attesa.
- Questo ciclo di attesa impedisce a tutti i thread coinvolti di procedere, creando una situazione di blocco permanente.

Il deadlock è una delle principali problematiche nei sistemi operativi multiprogrammati, poiché può portare a inefficienza e al blocco completo del sistema se non rilevato o gestito correttamente.

2 Tipi di Risorse e Concetti Base sui Deadlock

In un sistema multiprogrammato, le risorse disponibili sono limitate e organizzate in **tipi di risorsa**, ciascuno con un certo numero di *istanze* identiche. Ad esempio, un tipo di risorsa può essere una stampante, e ogni stampante disponibile rappresenta un'istanza di quel tipo.

2.1 Gestione delle Risorse da Parte dei Thread

Ogni thread deve seguire un ordine preciso per utilizzare le risorse:

1. **Richiedere la risorsa:** il thread invia una richiesta al sistema operativo per ottenere un'istanza disponibile della risorsa desiderata.
2. **Usare la risorsa:** una volta ottenuta, il thread può eseguire le operazioni necessarie.
3. **Rilasciare la risorsa:** dopo aver terminato, il thread restituisce l'istanza della risorsa al sistema, rendendola disponibile per altri thread.

Spiegazione: Questo ordine è fondamentale per evitare situazioni di conflitto. Richiedere risorse in modo casuale o trattenere risorse inutilmente aumenta la probabilità di deadlock.

2.2 Attesa per Risorse Non Disponibili

Se un thread richiede una risorsa di cui tutte le istanze sono già occupate, il thread deve entrare in **stato di attesa**.

Durante questo periodo:

- Il thread non può eseguire operazioni dipendenti dalla risorsa richiesta.
- Rimane sospeso fino a quando almeno un'istanza della risorsa diventa disponibile.

2.3 Deadlock e Dipendenze Circolari

Un **deadlock** si verifica quando ogni thread in un gruppo attende un evento che solo un altro thread dello stesso gruppo può generare.

Questi eventi si riferiscono tipicamente all'acquisizione o al rilascio di risorse condivise:

- Un thread potrebbe attendere che un altro thread rilasci una risorsa di cui ha bisogno.
- Se i thread formano un ciclo di attesa reciproca, nessuno può procedere, creando un blocco permanente.

Osservazione: La presenza di più istanze dello stesso tipo di risorsa può ridurre la probabilità di deadlock, ma non lo elimina completamente. La gestione corretta delle richieste e dei rilasci rimane essenziale per la sicurezza del sistema.

3 Deadlock con Più Mutex

Un caso classico di deadlock si verifica quando due thread condividono due **mutex** e li acquisiscono in ordine opposto. Questo scenario mostra chiaramente come la mancata coordinazione nell'accesso alle risorse possa portare a un blocco permanente.

3.1 Descrizione del Problema

Supponiamo due thread, chiamati `thread_one` e `thread_two`, che condividono due mutex: `first_mutex` e `second_mutex`.

- `thread_one` acquisisce prima `first_mutex` e poi tenta di acquisire `second_mutex`.
- `thread_two` acquisisce prima `second_mutex` e poi tenta di acquisire `first_mutex`.

Se ciascun thread riesce a bloccare la prima risorsa e poi prova a bloccare la seconda, entrambi rimangono in attesa dell'altra risorsa, generando un deadlock.

Osservazione: Questo problema si manifesta solo con determinati ordini di schedulazione dei thread. In alcuni casi, se un thread completa l'acquisizione e il rilascio dei mutex prima che l'altro inizi, il deadlock non si verifica.

3.2 Soluzione Sicura

Per evitare questo tipo di deadlock, è fondamentale imporre un **ordine coerente di acquisizione dei mutex** per tutti i thread. Ad esempio, tutti i thread devono acquisire prima `first_mutex` e poi `second_mutex`.

3.3 Esempio in C con pthreads

```
/* Creazione e inizializzazione dei mutex */
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;
pthread_mutex_init(&first_mutex, NULL);
pthread_mutex_init(&second_mutex, NULL);

/* thread_one esegue questa funzione */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);

    /* Eseguire alcune operazioni protette dai mutex */

    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);
}
```

```
/* thread_two esegue questa funzione */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);

    /* Eseguire alcune operazioni protette dai mutex */

    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);
}
```

3.4 Spiegazione Dettagliata dell'Esempio

- `pthread_mutex_lock(& mutex)` blocca il mutex specificato, impedendo ad altri thread di accedere contemporaneamente alla risorsa protetta.
- `pthread_mutex_unlock(& mutex)` libera il mutex, rendendo disponibile la risorsa agli altri thread.
- Nel nostro esempio, `thread_one` e `thread_two` possono bloccare ciascuno il primo mutex e poi rimanere bloccati tentando di acquisire il secondo, causando deadlock.
- La soluzione consiste nell'adottare un **ordine uniforme di acquisizione**: tutti i thread devono acquisire prima `first_mutex` e poi `second_mutex`, eliminando il ciclo di attesa.

Nota importante: Questa strategia è un esempio di regola generale per prevenire deadlock: se tutti i thread rispettano un ordine fisso per l'acquisizione delle risorse, è impossibile formare un ciclo di attesa.

4 Liveness e Fallimenti di Liveness

4.1 Concetto di Liveness

La **liveness** (o vitalità) è una proprietà fondamentale dei sistemi concorrenti. Essa garantisce che tutti i processi o thread facciano progressi durante la loro esecuzione, evitando che rimangano bloccati indefinitamente.

Spiegazione:

- Un sistema con liveness garantita permette a ogni processo di completare le proprie operazioni in un tempo finito.
- La liveness è complementare alla sicurezza (*safety*): mentre la sicurezza garantisce che il sistema non compia azioni errate, la liveness garantisce che esso effettivamente compia azioni desiderate.

4.2 Fallimento di Liveness

Un **liveness failure** (fallimento di vitalità) si verifica quando un processo non riesce a fare progressi e rimane in attesa indefinita.

Sintomi comuni di un fallimento di liveness:

- Prestazioni scadenti del sistema.
- Perdita significativa della reattività del sistema.

4.3 Esempio Semplice

Un esempio semplice di fallimento di liveness è un *loop infinito* all'interno di un thread o di un processo:

```
while (true) {
    // esegue operazioni senza mai terminare
}
```

In questo caso, il processo rimane in esecuzione per sempre, impedendo eventuali progressi e causando un fallimento di liveness.

4.4 Rischi legati alla Concorrenza

Nei sistemi concorrenti, l'uso di strumenti di mutua esclusione (**mutex**, **lock**, **semafori**) introduce il rischio di fallimenti di liveness:

- Se un thread detiene un mutex e non lo rilascia correttamente, altri thread possono rimanere bloccati in attesa.
- Anche se non si verifica un deadlock completo, ritardi o attese prolungate possono ridurre drasticamente la liveness del sistema.
- È quindi essenziale progettare algoritmi concorrenti che garantiscano sia la sicurezza che la vitalità dei processi.

Osservazione: La prevenzione dei fallimenti di liveness è un obiettivo critico nella programmazione concorrente, poiché migliora sia l'efficienza che la reattività del sistema.

5 Livelock

5.1 Definizione

Un **livelock** è una forma di fallimento di liveness in cui i thread cambiano continuamente il loro stato ma non fanno alcun progresso reale.

Spiegazione: A differenza di un deadlock, in un livelock i thread non sono bloccati: continuano ad eseguire operazioni, modificando il proprio stato, ma nessuno riesce a completare l'operazione desiderata. Questo produce inefficienza e blocco logico del sistema.

5.2 Somiglianza e Differenze con il Deadlock

- **Somiglianza:** sia deadlock che livelock impediscono ai thread di procedere con il loro lavoro.
- **Differenza chiave:** nel deadlock i thread sono bloccati e inattivi, mentre nel livelock i thread continuano a cambiare stato senza avanzare.

5.3 Analogia Intuitiva

Un'analogia comune per il livelock è la seguente: due persone cercano di attraversare un corridoio stretto e continuano a cedere l'una all'altra senza mai avanzare. Entrambe si muovono continuamente, ma nessuna raggiunge la destinazione.

5.4 Cause del Livelock

Il livelock si verifica tipicamente quando i thread:

- Ritentano continuamente un'operazione che inizialmente fallisce a causa di un'azione concorrente.
- Modificano il loro stato come risposta al conflitto, ma queste modifiche impediscono di completare l'operazione.

5.5 Strategia di Prevenzione

Una strategia efficace per prevenire il livelock consiste nell'introdurre un **ritardo casuale (random backoff)** prima di ritentare un'operazione fallita.

Spiegazione: Introducendo un piccolo intervallo di attesa casuale, i thread non tentano simultaneamente la stessa operazione nello stesso ordine, riducendo la probabilità di conflitti ripetuti e consentendo finalmente a uno dei thread di completare il compito.

6 Condizioni Necessarie per il Deadlock

Un deadlock può verificarsi solo se **tutte e quattro** le condizioni seguenti sono soddisfatte contemporaneamente. Queste condizioni sono fondamentali nella teoria dei sistemi operativi e sono note come le *quattro condizioni di Coffman*.

6.1 1. Mutua Esclusione (Mutual Exclusion)

- Almeno una risorsa deve essere **non condivisibile**.
- Solo un thread alla volta può utilizzare la risorsa.

Spiegazione: Se tutte le risorse fossero condivisibili tra i thread senza restrizioni, non sarebbe possibile che più thread si blocchino a vicenda. La mutua esclusione è quindi la base necessaria per creare conflitti sulle risorse.

6.2 2. Hold and Wait (Trattenimento e Attesa)

- Un thread detiene almeno una risorsa.
- Lo stesso thread sta contemporaneamente aspettando di acquisire altre risorse già detenute da altri thread.

Spiegazione: Questa condizione descrive situazioni in cui i thread non rilasciano le risorse già acquisite mentre attendono altre risorse. La combinazione di risorse detenute e risorse richieste è ciò che può generare un ciclo di attesa e quindi il deadlock.

6.3 3. Nessuna Preemption (No Preemption)

- Le risorse non possono essere rimosse forzatamente dal thread che le detiene.
- Una risorsa è rilasciata solo volontariamente dal thread che la possiede, generalmente dopo il completamento del compito associato.

Spiegazione: Se il sistema potesse forzare il rilascio delle risorse dai thread, i cicli di attesa potrebbero essere interrotti, impedendo il deadlock. La mancanza di preemption è quindi una condizione necessaria affinché il deadlock si realizzi.

6.4 4. Attesa Circolare (Circular Wait)

- Deve esistere una catena chiusa di thread in attesa, ad esempio $\{T_0, T_1, \ldots, T_n\}$.
- Ogni thread T_i sta aspettando una risorsa detenuta da T_{i+1}, con T_n che attende una risorsa detenuta da T_0.

Spiegazione: La presenza di una catena circolare di attesa è ciò che rende il deadlock irreversibile senza intervento esterno. Senza questa catena, i thread potrebbero eventualmente completare le proprie operazioni e rilasciare le risorse, evitando il blocco permanente.

Osservazione: Tutte e quattro le condizioni devono verificarsi contemporaneamente. Se anche una sola non è soddisfatta, il deadlock non può verificarsi. Questo principio è fondamentale per progettare sistemi operativi resilienti e prevenire il blocco dei thread.

7 Resource-Allocation Graph

7.1 Definizione e Scopo

Un **resource-allocation graph** (grafo di allocazione delle risorse) è uno strumento utile per descrivere e rilevare deadlock nei sistemi multiprogrammati.

Spiegazione: Il grafo permette di visualizzare le relazioni tra thread e risorse, mostrando quali risorse sono richieste, quali sono detenute e se esistono situazioni di blocco potenziale.

7.2 Tipi di Nodi

Il grafo utilizza due tipi di nodi:

- **Cerchi:** rappresentano i thread (T_1, T_2, ...).
- **Rettangoli:** rappresentano i tipi di risorsa (R_1, R_2, ...). All'interno dei rettangoli, i punti indicano le istanze disponibili per ciascun tipo di risorsa.

7.3 Tipi di Archi

- **Request edge ($\mathbf{T}_i \to \mathbf{R}_j$):** il thread sta aspettando di acquisire una risorsa.
- **Assignment edge ($\mathbf{R}_j \to \mathbf{T}_i$):** un'istanza della risorsa è stata allocata al thread.

Nota: Analizzando il grafo, è possibile rilevare deadlock osservando la presenza di cicli: un ciclo indica che i thread coinvolti stanno attendendo risorse in modo circolare e nessuno può procedere.

7.4 Lettura di un Resource-Allocation Graph

Esempio di grafo:

- **Thread:** T_1, T_2, T_3 (cerchi)
- **Risorse:** R_1, R_2, R_3, R_4 (rettangoli)
- T_1 è in attesa di R_1 e detiene R_2.
- T_2 detiene R_1 e R_2, ma è in attesa di R_3.
- T_3 detiene R_3 e non aspetta nulla.
- R_4 ha tre istanze libere e nessuna freccia.

Spiegazione passo passo:

1. Una freccia $T_i \rightarrow R_j$ indica che il thread non può proseguire finché l'istanza della risorsa non sarà disponibile.
2. Una freccia $R_j \rightarrow T_i$ indica che l'istanza della risorsa è attualmente in uso dal thread.
3. In questo esempio, osservando le relazioni tra T_1, T_2, T_3 e le risorse, non si forma alcun ciclo di attesa.

Osservazione: Poiché non esiste un ciclo chiuso di richieste tra i thread, non si verifica deadlock in questo scenario, anche se alcuni thread stanno aspettando risorse. Questo dimostra come un resource-allocation graph possa essere utilizzato per analizzare la presenza o meno di deadlock.

7.5 Rappresentazione Testuale del Grafo

```
T1 --> R1
R2 --> T1

T2 --> R3
R1 --> T2
R2 --> T2

T3 holds R3 (no request)

R4 has 3 free instances (no edges)
```

Spiegazione: Questa rappresentazione testuale mostra chiaramente chi attende cosa e chi detiene quale risorsa, rendendo possibile il rilevamento dei cicli senza un disegno grafico.

8 Cicli e Rilevazione del Deadlock

8.1 Cicli nel Resource-Allocation Graph

Il concetto di **ciclo** nel grafo di allocazione delle risorse è fondamentale per rilevare i deadlock.

- Se ogni tipo di risorsa ha **una sola istanza**, allora la presenza di un ciclo nel grafo indica **inequivocabilmente un deadlock**.
- In questo caso, un ciclo è sia **necessario** che **sufficiente** per confermare il deadlock.

Nota: Questo è il caso più semplice, spesso usato nei corsi introduttivi per spiegare i principi dei deadlock.

8.2 Cicli con Risorse Multiple

- Se un tipo di risorsa ha **più istanze**, la presenza di un ciclo non implica automaticamente un deadlock.
- Qui, il ciclo è **necessario** ma non **sufficiente** per determinare che un deadlock esiste.

Spiegazione: Quando più istanze sono disponibili, è possibile che un thread attenda una risorsa, ma un'altra istanza sia libera per un altro thread, permettendo al sistema di procedere senza blocchi.

8.3 Esempio di Deadlock con Cicli Multipli

Supponiamo che T_3 richieda R_2 ma nessuna istanza sia disponibile. Si aggiunge un arco $T_3 \to R_2$, creando due cicli nel grafo.

- T_2 attende R_3 (detenuta da T_3).
- T_3 attende R_2 (detenuta da T_1 o T_2).
- T_1 attende R_1 (detenuta da T_2).

Spiegazione: Ogni thread attende una risorsa che solo un altro thread in attesa può rilasciare. Nessuno dei thread può proseguire, generando un deadlock completo.

8.4 Esempio di Ciclo Senza Deadlock

Consideriamo un grafo che contiene un ciclo: $T_1 \to R_1 \to T_3 \to R_2 \to T_1$.

- T_4 può rilasciare un'istanza di R_2, permettendo a T_3 di acquisirla.
- Una volta ottenuta R_2, T_3 completa la propria esecuzione e rilascia R_1.
- Il rilascio di R_1 consente a T_1 di procedere, spezzando completamente il ciclo.

Spiegazione: In questo caso, il ciclo non provoca deadlock perché il sistema dispone di istanze libere sufficienti e alcuni thread possono liberare risorse, permettendo agli altri di avanzare.

8.5 Osservazioni Generali

- Se il resource-allocation graph **non contiene cicli**, il sistema è garantito essere **privo di deadlock**.
- La presenza di cicli richiede un'analisi più approfondita, soprattutto quando le risorse hanno più istanze, per determinare se un deadlock effettivamente esiste.

Nota didattica: La distinzione tra cicli con una sola istanza e cicli con più istanze è fondamentale per capire perché non tutti i cicli indicano necessariamente un deadlock.

9 Strategie di Gestione dei Deadlock

Esistono tre strategie principali per gestire i deadlock nei sistemi multiprogrammati:

9.1 1. Ignorare i Deadlock

- Questo approccio assume che il **ciclo di attesa** non si verificherà mai.
- È noto come la politica del *ostrich algorithm*, "mettere la testa sotto la sabbia" di fronte al problema.

Spiegazione: Questa strategia è semplice e non richiede overhead aggiuntivo, ma rischia che, in caso di deadlock reale, il sistema subisca un degrado delle prestazioni senza che ci sia alcuna correzione automatica.

9.2 2. Prevenzione dei Deadlock

- La prevenzione consiste nell'eliminare almeno una delle quattro condizioni necessarie per il deadlock (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait).

Esempi di tecniche di prevenzione:

- Consentire la condivisione di risorse quando possibile (riducendo la mutual exclusion).
- Richiedere tutte le risorse necessarie a un thread prima che inizi l'esecuzione (evitando hold and wait).
- Consentire la preemption forzata di risorse se un thread ne detiene troppo a lungo (superando no preemption).
- Imponendo un ordine fisso di acquisizione delle risorse per tutti i thread (evitando circular wait).

Osservazione: La prevenzione è efficace, ma può limitare la flessibilità e ridurre l'utilizzo ottimale delle risorse.

9.3 3. Evitamento dei Deadlock

- L'evitamento richiede di conoscere in anticipo i bisogni di risorse di ciascun thread.
- Il sistema può decidere se una richiesta di risorsa è **sicura** da concedere senza generare un ciclo potenzialmente deadlockante.

Spiegazione: L'algoritmo del banchiere di Dijkstra è un esempio classico di avoidance: il sistema valuta se, dopo aver concesso la risorsa, rimarrebbero abbastanza risorse per permettere a tutti i thread di completare il loro lavoro. Solo se la richiesta è sicura viene concessa.

9.4 4. Rilevazione e Recupero

- Questa strategia accetta che i deadlock possano verificarsi, ma garantisce che il sistema possa **rilevarli** e **risolverli**.

Spiegazione:

- La rilevazione può avvenire tramite un controllo periodico del resource-allocation graph alla ricerca di cicli.
- Il recupero consiste nel terminare o preemptare alcuni thread coinvolti nel deadlock per liberare le risorse.

9.5 Osservazioni Generali

- Senza rilevazione o recupero, i deadlock possono degradare silenziosamente le prestazioni complessive del sistema.
- La scelta della strategia dipende dal tipo di sistema: sistemi real-time, di batch o general-purpose richiedono approcci differenti per bilanciare sicurezza, efficienza e overhead.

Nota didattica: Comprendere le tre strategie principali e i loro compromessi è fondamentale per progettare sistemi concorrenti robusti e affidabili.

10 Prevenzione dei Deadlock: Mutua Esclusione

10.1 Principio Generale

Il deadlock può essere prevenuto assicurandosi che almeno una delle condizioni necessarie per il deadlock **non si verifichi mai**.

La prima condizione considerata è la **mutua esclusione**.

10.2 Mutua Esclusione

- Alcune risorse non possono essere condivise tra più thread simultaneamente.
- Risorse condivisibili, come file di sola lettura, non causano deadlock perché possono essere accedute da più thread contemporaneamente.

Spiegazione: Se una risorsa è naturalmente condivisibile, i thread non devono mai attendere per accedervi. In questo caso, la mutua esclusione non rappresenta un rischio di deadlock.

10.3 Limiti della Prevenzione tramite Mutua Esclusione

- Molte risorse sono intrinsecamente non condivisibili, come stampanti, dispositivi di I/O o sezioni critiche della memoria.
- La mutua esclusione non può quindi essere completamente eliminata.

Osservazione didattica: Poiché non è possibile eliminare completamente la mutua esclusione per le risorse non condivisibili, **la prevenzione del deadlock non può basarsi unicamente su questa condizione**. Altre condizioni devono essere gestite per ridurre il rischio di blocco dei thread.

10.4 Esempio Pratico

- File di sola lettura: più thread possono leggere simultaneamente senza attesa. Nessun deadlock possibile.
- Stampante condivisa: solo un thread alla volta può usarla. La mutua esclusione è inevitabile e da sola non permette di prevenire il deadlock.

11 Prevenzione dei Deadlock: Hold-and-Wait

11.1 Principio Generale

Un approccio alla prevenzione dei deadlock consiste nell'eliminare la condizione di **Hold-and-Wait** (trattenimento e attesa).

Spiegazione: La condizione Hold-and-Wait si verifica quando un thread detiene già una o più risorse mentre ne richiede altre, creando il rischio di attese circolari e deadlock.

11.2 Strategie per Eliminare Hold-and-Wait

Due opzioni principali:

1. **Richiesta preventiva di tutte le risorse:** Un thread deve richiedere tutte le risorse di cui avrà bisogno prima di iniziare l'esecuzione. In questo modo, non terrà risorse mentre ne attende altre, eliminando Hold-and-Wait.

2. **Richiesta solo quando non si possiedono risorse:** Un thread può richiedere risorse soltanto se al momento non detiene nessuna risorsa. Così si evita di trattenere risorse mentre si è in attesa di altre.

11.3 Limiti di queste Strategie

- **Riduzione dell'efficienza:** le risorse potrebbero rimanere allocate ma inutilizzate fino a quando il thread non ottiene tutto ciò che serve.

- **Attesa indefinita:** i thread potrebbero rimanere bloccati a lungo se devono rispettare restrizioni di richiesta preventiva, causando ritardi anche senza deadlock.

Osservazione didattica: La prevenzione tramite eliminazione di Hold-and-Wait è efficace nel prevenire deadlock, ma introduce compromessi in termini di efficienza e utilizzo ottimale delle risorse. È quindi una strategia utile in sistemi in cui la sicurezza e la stabilità sono più importanti della performance massima.

11.4 Esempio Pratico

- Un thread che deve stampare e scrivere su un file: Può richiedere *prima* la stampante e il file contemporaneamente, evitando di trattenere una risorsa mentre attende l'altra.

- In alternativa, può richiedere la stampante solo se non detiene il file, e viceversa.

12 Prevenzione dei Deadlock: No-Preemption (Preemption delle Risorse)

12.1 Principio Generale

Un approccio alla prevenzione dei deadlock consiste nell'eliminare la condizione di **No-Preemption**.

Spiegazione: La condizione No-Preemption afferma che le risorse non possono essere rimosse forzatamente dai thread che le detengono. Se questa condizione viene rimossa, è possibile prevenire deadlock prendendo risorse da thread che le detengono quando necessario.

12.2 Strategia di Preemption delle Risorse

Quando un thread richiede una risorsa non disponibile:

1. Tutte le risorse attualmente detenute dal thread vengono **preempted** (riportate al sistema).
2. Il thread entra in attesa finché non può acquisire **tutte le risorse necessarie**.

12.3 Alternativa: Preemption Selettiva

- Se le risorse richieste sono detenute da un thread già bloccato, esse possono essere preempted e restituite al sistema.
- Se le risorse richieste sono detenute da un thread in esecuzione, il thread richiedente deve attendere.

Nota didattica: Questa strategia assicura che un thread riprenda l'esecuzione solo quando riesce a ottenere tutte le risorse necessarie, prevenendo potenziali deadlock.

12.4 Esempio Pratico

- Un thread A detiene la stampante e richiede il file.
- Se il file è detenuto da un thread bloccato B, il sistema può preemptare il file da B e assegnarlo ad A.
- Se invece il file è detenuto da un thread C in esecuzione, A deve attendere fino al rilascio.
- Una volta che A ottiene sia la stampante sia il file, può riprendere l'esecuzione.

Osservazione: La preemption delle risorse aumenta la complessità del sistema, ma permette di rompere cicli di attesa e prevenire deadlock senza dover rinunciare alla mutua esclusione o all'hold-and-wait in modo drastico.

13 Prevenzione dei Deadlock: Eliminazione della Circular Wait

13.1 Principio Generale

Un approccio alla prevenzione dei deadlock consiste nell'eliminare la condizione di **Circular Wait** (attesa circolare).

Spiegazione: La condizione Circular Wait si verifica quando esiste una catena chiusa di thread in attesa di risorse, ciascuno bloccato da quello successivo. Rompere questa catena impedisce completamente il verificarsi di deadlock.

13.2 Strategia: Ordinamento Globale delle Risorse

1. Si assegna un **ordine globale** a tutti i tipi di risorse del sistema.
2. Indichiamo l'insieme delle risorse come $R = \{R_1, R_2, \ldots, R_m\}$ e definiamo una funzione $F : R \to \mathbb{N}$ che stabilisce l'ordine crescente delle risorse.
3. I thread possono richiedere solo le risorse necessarie, ma **sempre in ordine crescente secondo F**.
4. Un thread non può richiedere una risorsa con ordine maggiore prima di aver acquisito tutte le risorse di ordine inferiore.

13.3 Motivazione e Spiegazione

- Poiché tutte le richieste seguono la stessa direzione crescente, non è possibile formare un ciclo di attesa tra thread.
- Questo spezza automaticamente la condizione necessaria di Circular Wait, prevenendo i deadlock senza dover eliminare mutua esclusione, hold-and-wait o preemption.

13.4 Esempio Pratico

- Supponiamo tre risorse R_1, R_2, R_3 con ordine crescente F(R_1) ¡ F(R_2) ¡ F(R_3).
- Un thread che necessita di R_2 e R_3 deve prima richiedere R_2, poi R_3.
- Nessun altro thread può acquisire R_3 prima di R_2 secondo l'ordine globale, quindi non può crearsi una catena circolare.
- Di conseguenza, non si verifica alcun deadlock.

Osservazione didattica: L'assegnazione di un ordine globale è semplice da implementare e molto efficace, ma richiede che tutti i thread conoscano l'ordine e lo rispettino rigorosamente.

14 Deadlock Avoidance: Idea di Base

14.1 Principio Generale

L'**evitamento del deadlock** si differenzia dalla prevenzione in quanto non limita rigidamente le condizioni del sistema, ma richiede informazioni aggiuntive sui bisogni futuri dei thread.

Spiegazione: Mentre la prevenzione elimina alcune condizioni necessarie al deadlock (mutua esclusione, hold-and-wait, no-preemption, circular wait), può ridurre l'efficienza. L'evitamento, invece, cerca di concedere risorse in modo sicuro, ottimizzando l'uso delle risorse e prevenendo i deadlock solo quando necessario.

14.2 Informazioni sui Bisogni Futuri

- Ogni thread deve fornire informazioni aggiuntive riguardanti il **futuro utilizzo delle risorse**, ossia l'ordine e la quantità di richieste e rilascio.
- Conoscere questo ordine permette al sistema di decidere se soddisfare immediatamente una richiesta o posticiparla per mantenere lo stato sicuro.

14.3 Monitoraggio delle Risorse

Il sistema tiene traccia di tre elementi principali:

1. **Risorse disponibili** attualmente libere nel sistema.
2. **Allocazioni correnti** di risorse ai thread.
3. **Domanda futura** massima di ciascun thread per ciascun tipo di risorsa.

Spiegazione: Questi dati permettono di simulare l'esito del soddisfacimento di una richiesta e determinare se il sistema rimane in uno **stato sicuro** (safe state) dove tutti i thread possono eventualmente completare la loro esecuzione senza deadlock.

14.4 Dichiarazione della Massima Necessità

- Ogni thread dichiara la **massima quantità** di ogni tipo di risorsa di cui avrà bisogno durante la sua esecuzione.
- Questa dichiarazione consente al sistema di pianificare e decidere se concedere la risorsa richiesta in modo sicuro.

14.5 Verifica dello Stato Sicuro

- Il sistema controlla se la concessione di una richiesta mantiene il sistema in uno stato sicuro.
- Solo se lo stato resta sicuro, la richiesta viene soddisfatta; altrimenti, il thread deve attendere.

Osservazione didattica: L'evitamento del deadlock permette di mantenere alte prestazioni senza rinunciare alla sicurezza, ma richiede che i thread siano cooperativi e dichiarino correttamente i propri bisogni massimi.

15 Safe State e Safe Sequences

15.1 Definizione di Safe State

Un sistema è considerato **safe** (sicuro) se esiste almeno una sequenza di thread $\langle T_1, T_2, \ldots, T_n \rangle$ tale che tutti i thread possano completare la loro esecuzione senza causare deadlock.

Spiegazione: La sicurezza del sistema non significa che non ci siano attese, ma che **è possibile trovare un ordine di esecuzione che consente a tutti i thread di terminare correttamente**.

15.2 Notazione delle Sequenze

- T_i: thread valutato alla posizione i nella sequenza sicura.
- T_j: thread con $j < i$, cioè quelli che appaiono prima di T_i nella sequenza.

15.3 Condizione per la Sicurezza

Il thread T_i può completare la propria esecuzione se le sue necessità di risorse possono essere soddisfatte utilizzando:

1. Le risorse **attualmente libere** nel sistema.
2. Le risorse che saranno **rilasciate dai thread** T_j precedenti nella sequenza.

Spiegazione: In altre parole, T_i può attendere che i thread precedenti T_j completino l'esecuzione e rilascino le risorse necessarie. Una volta ottenute tutte le risorse necessarie, T_i completa il suo lavoro e libera le risorse per i thread successivi.

15.4 Determinazione dello Stato Sicuro

- Se **esiste almeno un ordine** dei thread che permette a tutti di completare, il sistema è in uno **stato sicuro**.
- Se **nessun ordine** consente a tutti i thread di terminare, il sistema è in uno **stato non sicuro** (*unsafe state*), che potrebbe portare a deadlock.

15.5 Osservazioni Didattiche

- Lo stato sicuro è un concetto teorico fondamentale per il **deadlock avoidance**.
- Non implica che i thread siano attualmente in esecuzione senza attese, ma che esiste un **percorso garantito di completamento** per tutti.
- Il concetto di safe sequence aiuta a progettare algoritmi come il *Banker's Algorithm*, che verifica le richieste di risorse simulando stati sicuri.

16 Safe vs Unsafe States

16.1 Definizione di Safe State

- Uno **safe state** garantisce che il sistema non possa entrare in deadlock.
- Tutti i thread possono completare la loro esecuzione seguendo almeno una **safe sequence**.

16.2 Definizione di Unsafe State

- Se si verifica un deadlock, il sistema deve essere in uno **unsafe state**.
- Uno **stato non sicuro** non significa che il deadlock sia già avvenuto, ma solo che esiste la possibilità che si verifichi.
- In uno stato non sicuro, il sistema operativo non può controllare completamente le future richieste dei thread.

16.3 Osservazioni Didattiche

- In **safe states**, il sistema può sempre evitare evoluzioni pericolose, concedendo risorse solo se il sistema rimane sicuro.
- In **unsafe states**, il verificarsi di un deadlock dipende interamente dal comportamento futuro dei thread.
- Questo concetto è cruciale per algoritmi di **deadlock avoidance** come il *Banker's Algorithm*.

16.4 Esempio di Safe State

Scenario:

- Sistema con un totale di 12 risorse.
- Tre thread: T_0, T_1, T_2.

Safe Sequence: $\langle T_1, T_0, T_2 \rangle$

1. T_1 completa per primo $\rightarrow$ rilascia le risorse $\rightarrow$ 5 risorse ora disponibili.
2. T_0 completa successivamente $\rightarrow$ rilascia le sue risorse $\rightarrow$ 10 risorse disponibili.
3. T_2 completa per ultimo $\rightarrow$ tutte le 12 risorse sono nuovamente disponibili.

Spiegazione: Questa sequenza mostra chiaramente che tutti i thread possono completare senza causare deadlock. Lo stato iniziale è quindi sicuro (**safe state**) perché esiste almeno una sequenza di esecuzione che garantisce il completamento di tutti i thread.

17 Transizione da Safe a Unsafe State

17.1 Scenario di Transizione

Consideriamo un sistema con tre thread T_0, T_1, T_2 e un numero totale di risorse. Supponiamo che inizialmente il sistema sia in uno **safe state**.

- Al tempo t_1, il thread T_2 richiede un'ulteriore risorsa.
- Il sistema concede la richiesta $\rightarrow$ lo stato diventa **unsafe**.

17.2 Conseguenze della Transizione

- In questa nuova configurazione, solo T_1 può ancora completare la sua esecuzione.
- Dopo che T_1 termina, vengono rilasciate solo 4 risorse.
- T_0 potrebbe richiedere 5 risorse $\rightarrow$ nessuna risorsa disponibile $\rightarrow$ deve attendere.
- T_2 potrebbe richiedere fino a 6 risorse $\rightarrow$ anch'esso bloccato.
- Risultato: sia T_0 che T_2 diventano bloccati $\rightarrow$ si verifica deadlock.

17.3 Causa del Deadlock

- La causa principale è stata il **consentire a T_2 di ottenere una risorsa extra al tempo t_1**.
- Se T_2 avesse atteso fino al rilascio di altre risorse, il deadlock sarebbe potuto essere evitato.

17.4 Spiegazione Didattica

- Questo esempio mostra chiaramente come uno stato **safe** possa diventare **unsafe** concedendo risorse senza controllo.
- Evidenzia l'importanza di verificare sempre che ogni richiesta non porti il sistema in uno stato non sicuro prima di assegnare risorse.
- Concetto chiave per algoritmi di **deadlock avoidance**: *non concedere risorse se lo stato risultante non è sicuro.*

17.5 Nota Esemplificativa

In pratica: algoritmi come il *Banker's Algorithm* simulano le assegnazioni future e determinano se concedere una risorsa mantiene il sistema in uno stato sicuro. Questo evita transizioni che potrebbero generare deadlock anche se il sistema era inizialmente sicuro.

18 Safe State e Deadlock Avoidance

18.1 Principio Generale

Gli algoritmi di **deadlock avoidance** si basano sul concetto di **safe state**.

- Il sistema deve sempre rimanere in uno stato sicuro.
- Quando un thread richiede una risorsa disponibile, il sistema verifica se concederla mantiene lo stato sicuro.
- La richiesta viene accettata solo se la sicurezza del sistema è preservata.

18.2 Gestione delle Risorse Disponibili

- Una risorsa può essere tecnicamente libera ma **non concessa immediatamente** se assegnarla rischia di far entrare il sistema in uno stato non sicuro.
- Questa decisione preventiva riduce il rischio di deadlock, anche a costo di un utilizzo globale delle risorse meno efficiente.

18.3 Spiegazione Didattica

- Il concetto chiave è che il sistema valuta **tutte le possibili conseguenze** di una richiesta di risorsa prima di concederla.
- Questo garantisce che ogni thread abbia la possibilità di completare la propria esecuzione senza rischiare deadlock.
- La strategia dimostra un compromesso tra sicurezza (evitare deadlock) e performance (minore utilizzo immediato delle risorse).

18.4 Collegamento con Safe State

- La definizione di **safe state** è centrale: solo se il sistema rimane safe, una richiesta viene soddisfatta.
- Questo approccio è alla base di algoritmi come il *Banker's Algorithm*, che simula le assegnazioni future per determinare la sicurezza del sistema.

Nota didattica: Anche se una risorsa è disponibile, la sua concessione può essere posticipata. Questa strategia preventiva è il cuore dell'evitamento del deadlock e permette al sistema di operare senza rischi di blocco circolare tra i thread.

19 Prevenzione dei Deadlock con Claim Edges

19.1 Principio Generale

La prevenzione del deadlock può essere gestita mediante l'uso di **claim edges** nei grafi di allocazione delle risorse.

- Ogni tipo di risorsa ha **una sola istanza**.
- Un **claim edge** $T_i \to R_j$ indica che il thread T_i potrebbe richiedere in futuro la risorsa R_j.
- I claim edges sono rappresentati come linee tratteggiate e devono essere dichiarati prima dell'inizio dell'esecuzione del thread.

19.2 Conversione in Request Edge

- Quando il thread T_i effettua una richiesta per R_j, il claim edge diventa un **request edge** (freccia solida).
- La richiesta viene concessa solo se convertirla in **assignment edge** (risorsa assegnata) non genera un ciclo nel grafo.

19.3 Gestione dei Cicli

- Se la conversione in assignment edge creerebbe un **ciclo**, il sistema entra in uno **stato non sicuro** e il thread deve attendere.
- Un ciclo indica che **un futuro deadlock è possibile**, anche se la risorsa richiesta è attualmente libera.

19.4 Spiegazione Didattica

- Questo metodo è una forma preventiva di deadlock avoidance.
- I claim edges permettono al sistema di **simulare le richieste future** e di rifiutare temporaneamente quelle che potrebbero portare a stati insicuri.
- L'approccio garantisce che i thread non entrino in situazioni di deadlock, pur consentendo un utilizzo più flessibile delle risorse rispetto alla semplice prevenzione classica.

19.5 Esempio Concettuale

- Supponiamo tre thread T_1, T_2, T_3 e tre risorse R_1, R_2, R_3.
- Claim edges: $T_1 \dashrightarrow R_1$, $T_2 \dashrightarrow R_2$, $T_3 \dashrightarrow R_3$.
- Quando T_1 richiede R_1, il sistema verifica che assegnarla non crei un ciclo nel grafo.
- Se un ciclo apparisse, T_1 deve attendere fino a quando l'assegnazione non è sicura.

20 Banker's Algorithm

20.1 Overview

Il **Banker's Algorithm** è utilizzato per la **deadlock avoidance** in sistemi con più istanze per tipo di risorsa, dove i grafi di allocazione delle risorse non sono sufficienti.

Idea: Il sistema si comporta come un banchiere prudente, che non concede mai più risorse di quante possano essere rilasciate senza rischiare insolvenza (deadlock).

- Ogni thread dichiara in anticipo il numero massimo di istanze di ciascun tipo di risorsa di cui potrà aver bisogno.
- Le quantità massime dichiarate non possono superare il totale disponibile nel sistema.
- Quando un thread richiede risorse, il sistema verifica se concederle mantiene il sistema in uno **stato sicuro**.
- Se lo stato rimane sicuro → la richiesta è approvata.
- Se concedere le risorse porta a uno stato non sicuro → il thread deve attendere.

20.2 Strutture Dati Richieste

- $Available[j]$: numero di istanze libere della risorsa R_j.
- $Max[i, j]$: numero massimo di istanze di R_j che il thread T_i può richiedere.
- $Allocation[i, j]$: numero di istanze di R_j attualmente assegnate a T_i.
- $Need[i, j]$: numero di ulteriori istanze di R_j che T_i può ancora richiedere per completare l'esecuzione.

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

20.3 Safety Algorithm (Verifica Stato Sicuro)

Definiamo i vettori:

- $Work[m]$: risorse temporaneamente disponibili per la simulazione.
- $Finish[n]$: stato di completamento dei thread.

Passi:

1. Inizializza: $Work = Available$ $Finish[i] =$ false per tutti i thread i
2. Trova un thread i tale che: $Finish[i] =$ false e $Need[i] \leq Work$
3. Se trovato, simula il completamento di T_i: $Work = Work + Allocation[i]$ $Finish[i] =$ true
4. Ripeti la ricerca finché non si trovano più thread che soddisfano la condizione.
5. Se $Finish[i] =$ true per tutti i thread, lo stato è **safe**. Altrimenti, lo stato è **unsafe**.

20.4 Gestione di una Richiesta di Risorsa

Quando un thread T_i richiede risorse, l'algoritmo procede in quattro passaggi:

1. **Verifica della necessità dichiarata:** Accetta la richiesta solo se $Request_i \leq Need_i$. Se $Request_i$ eccede Max_i, viene segnalato un errore.

2. **Verifica della disponibilità:** Se $Request_i \leq Available$, continua. Altrimenti, T_i deve attendere.

3. **Allocazione provvisoria:**

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

4. **Test di sicurezza:** Se il nuovo stato è **safe** → concedi la richiesta. Se il nuovo stato è **unsafe** → nega la richiesta e ripristina i valori originali.

20.5 Esempio Numerico: Safety Check

- Risorse totali: 10
- Risorse allocate: 2
- Risorse inizialmente disponibili: 8

Spiegazione: Il sistema simula se tutti i thread possono completare in una sequenza safe. Solo se esiste una sequenza valida che consente a tutti i thread di terminare, la richiesta viene approvata.

Nota didattica: L'algoritmo garantisce che il sistema rimanga sempre in uno **stato sicuro**, prevenendo deadlock anche in presenza di più istanze di ciascun tipo di risorsa.

20.6 Esempio Pratico: Applicazione del Banker's Algorithm

20.6.1 Dati Iniziali

Supponiamo un sistema con 3 thread (T_0, T_1, T_2) e 3 tipi di risorse (R_0, R_1, R_2).

- Risorse totali disponibili: $R_0 = 10$, $R_1 = 5$, $R_2 = 7$
- Allocazioni correnti e massime richieste (Max) dei thread:

20.6.2 Tabella delle Risorse

Thread	Allocation R0	Allocation R1	Allocation R2	Max R0	Max R1	Max R2	Need R0	Neec
T_0	0	1	0	7	5	3	7	4
T_1	2	0	0	3	2	2	1	2
T_2	3	0	2	9	0	2	6	0

20.6.3 Calcolo di Need

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

- T_0: Need = (7-0, 5-1, 3-0) = (7,4,3)
- T_1: Need = (3-2, 2-0, 2-0) = (1,2,2)
- T_2: Need = (9-3, 0-0, 2-2) = (6,0,0)

20.6.4 Risorse Disponibili Iniziali

$$Available = Total - \sum Allocation = (10 - 5, 5 - 1, 7 - 2) = (5, 4, 5)$$

20.6.5 Verifica dello Stato Sicuro (Safety Check)

Step 1: Trova un thread T_i con $Need[i] \leq Available$

- T_1: Need = (1,2,2) $\leq$ Available = (5,4,5) $\rightarrow$ sì
- Simula completamento di T_1:

$$Available = Available + Allocation[T_1] = (5, 4, 5) + (2, 0, 0) = (7, 4, 5)$$

$Finish[T_1] = true$

Step 2: Trova un altro thread T_i non finito con Need $\leq$ Available

- T_0: Need = (7,4,3) $\leq$ Available = (7,4,5) $\rightarrow$ sì
- Simula completamento di T_0:

$$Available = Available + Allocation[T_0] = (7, 4, 5) + (0, 1, 0) = (7, 5, 5)$$

$Finish[T_0] = true$

Step 3: Trova un altro thread T_i non finito

- T_2: Need = (6,0,0) $\leq$ Available = (7,5,5) $\rightarrow$ sì
- Simula completamento di T_2:

$$Available = Available + Allocation[T_2] = (7, 5, 5) + (3, 0, 2) = (10, 5, 7)$$

$Finish[T_2] = true$

20.6.6 Conclusione

Tutti i thread hanno completato → lo stato iniziale è **safe**.

Safe Sequence:

$$\langle T_1, T_0, T_2 \rangle$$

Spiegazione didattica: Questo esempio mostra passo passo come il Banker's Algorithm simuli il completamento dei thread e determini che il sistema può concedere le risorse senza causare deadlock.

21 Rilevamento dei Deadlock e Recovery

21.1 Introduzione

Se un sistema non adotta strategie di prevenzione o evitamento, i **deadlock** possono verificarsi. Per gestirli, il sistema deve fornire due strumenti fondamentali:

1. **Deadlock Detection** (Rilevamento): analizzare lo stato corrente del sistema per verificare se esiste un deadlock.
2. **Deadlock Recovery** (Recupero): ripristinare il sistema una volta che un deadlock è stato rilevato.

21.2 Applicabilità

- Questi metodi funzionano sia per sistemi con **risorse a singola istanza** che con **risorse a più istanze**.
- Introducono però un certo **overhead**:
  - Memoria necessaria per memorizzare le informazioni sullo stato del sistema.
  - Costi computazionali per eseguire l'algoritmo di rilevamento del deadlock.

21.3 Considerazioni sul Recupero

- Il recupero dai deadlock può comportare **perdita di dati**, poiché alcuni processi potrebbero dover essere terminati forzatamente per liberare risorse.
- La scelta dei processi da terminare o delle risorse da prelevare deve essere fatta con attenzione per ridurre al minimo l'impatto sul sistema.

21.4 Spiegazione Didattica

- La rilevazione e il recupero sono soluzioni **reattive**, in contrasto con le strategie preventive o di avoidance.
- L'approccio reattivo garantisce che i deadlock vengano gestiti quando si verificano, ma comporta costi maggiori in termini di risorse di sistema e possibile interruzione dei processi.
- È quindi essenziale bilanciare **costo dell'overhead** e **rischio di deadlock** nella progettazione del sistema operativo.

22 Rilevamento dei Deadlock con Wait-For Graphs

22.1 Introduzione

Il **wait-for graph** è uno strumento utilizzato per il rilevamento dei deadlock in sistemi in cui ogni tipo di risorsa ha **una sola istanza**.

22.2 Costruzione del Wait-For Graph

1. Partendo dal **resource-allocation graph**, rimuovere tutti i nodi relativi alle risorse.
2. Rimangono solo i nodi corrispondenti ai thread.
3. Aggiungere un **arco** $T_i \to T_j$ se T_i è in attesa di una risorsa attualmente posseduta da T_j.

22.3 Interpretazione

- Ogni arco rappresenta una situazione di attesa tra thread.
- Un **ciclo** nel wait-for graph indica che i thread coinvolti sono bloccati in un deadlock.
- In assenza di cicli, il sistema è libero da deadlock.

22.4 Esempio Concettuale

- Tre thread T_0, T_1, T_2 e tre risorse R_0, R_1, R_2, una sola istanza ciascuna.
- Supponiamo:
  - T_0 detiene R_0 e aspetta $R_1 \to$ arco $T_0 \to T_1$
  - T_1 detiene R_1 e aspetta $R_2 \to$ arco $T_1 \to T_2$
  - T_2 detiene R_2 e aspetta $R_0 \to$ arco $T_2 \to T_0$
- Si forma un ciclo $T_0 \to T_1 \to T_2 \to T_0 \to$ deadlock.

22.5 Spiegazione Didattica

- La costruzione del wait-for graph semplifica l'analisi del deadlock, eliminando le risorse dal grafo.
- Questo metodo è particolarmente efficiente per sistemi con risorse a singola istanza.
- Il sistema può **controllare periodicamente il grafo** per rilevare cicli e, quindi, deadlock.

22.6 Nota Operativa

- La rilevazione dei deadlock tramite wait-for graph è una tecnica **reattiva**: identifica i deadlock già in corso, ma non li previene.
- Una volta rilevato un ciclo, il sistema può intervenire con procedure di **recovery** come terminazione di thread o preemption di risorse.

23 Rilevamento dei Deadlock con Risorse Multiple

23.1 Introduzione

Il metodo del **wait-for graph** non funziona quando i tipi di risorsa hanno **più istanze**. Per questi sistemi è necessario utilizzare un **algoritmo dinamico di rilevamento dei deadlock**, simile alle strutture del *Banker's Algorithm*.

23.2 Strutture Dati Necessarie

- $Available[m]$: vettore che indica il numero di istanze libere per ciascun tipo di risorsa.
- $Allocation[n][m]$: matrice che indica le risorse attualmente allocate a ciascun thread.
- $Request[n][m]$: matrice che indica le richieste correnti di ciascun thread.

Nota: Le righe $Allocation_i$ e $Request_i$ sono trattate come vettori, e si utilizza la comparazione vettoriale $\leq$ per verificare se le richieste possono essere soddisfatte con le risorse disponibili.

23.3 Algoritmo di Rilevamento Deadlock (Multiple Resource Instances)

Definiamo due vettori:

- $Work[m]$: inizializzato con $Available$
- $Finish[n]$: indica se un thread può completare; inizializzato in base alle allocazioni correnti:

$$Finish[i] = \begin{cases} false & \text{se } Allocation_i \neq 0 \\ true & \text{se } Allocation_i = 0 \end{cases}$$

23.4 Passi dell'Algoritmo

1. Cerca ripetutamente un thread T_i tale che:

$$Finish[i] = false \quad \text{e} \quad Request_i \leq Work$$

2. Se tale thread è trovato:
   - Simula il completamento: $Work = Work + Allocation_i$
   - Imposta $Finish[i] = true$
   - Continua la ricerca per altri thread
3. Alla fine:
   - Se esiste almeno un $Finish[i] = false$, i thread corrispondenti sono **in deadlock**.
   - Se tutti i Finish[i] = true, nessun deadlock è presente.

23.5 Spiegazione Didattica

- L'algoritmo simula se tutti i thread attivi possono eventualmente completare, assegnando virtualmente le risorse disponibili ai thread che possono soddisfare le loro richieste.
- Se non esiste una sequenza di completamento per tutti i thread, allora un deadlock è presente.
- Questo metodo è l'equivalente dinamico del Banker's Algorithm per la rilevazione dei deadlock, senza bisogno di conoscenze a priori sulle richieste massime dei thread.

23.6 Nota Operativa

- Questa tecnica è più generale del wait-for graph e funziona anche con sistemi complessi dove più thread condividono più istanze dello stesso tipo di risorsa.
- L'analisi introduce overhead computazionale, ma permette di rilevare deadlock anche quando i metodi statici non sono sufficienti.

24 Perché le Risorse Sono Rilasciate Immediatamente nell'Algoritmo di Rilevamento

24.1 Principio Generale

Durante l'esecuzione dell'algoritmo di rilevamento dei deadlock:

- Se $Request_i \leq Work$, si assume che T_i non faccia parte di un deadlock.
- L'algoritmo fa un'ipotesi ottimistica:
  - T_i non richiederà più risorse di quelle già dichiarate.
  - T_i può completare l'esecuzione e rilasciare tutte le risorse detenute.
- Pertanto, le risorse attualmente allocate a T_i vengono aggiunte immediatamente a $Work$.
- Se questa ipotesi risultasse errata e T_i causasse un deadlock in seguito, la prossima esecuzione dell'algoritmo rileverà il deadlock.

24.2 Esempio Numerico

Consideriamo un sistema con 3 thread (T_0, T_1, T_2) e 3 tipi di risorsa (R_0, R_1, R_2) con più istanze.

- **Available**: $(3, 2, 2)$
- **Allocation**:
$$\begin{bmatrix} 0 & 1 & 0 \\ 2 & 0 & 0 \\ 3 & 0 & 2 \end{bmatrix}$$
- **Request** (richieste correnti dei thread):
$$\begin{bmatrix} 7 & 4 & 3 \\ 1 & 2 & 2 \\ 6 & 0 & 0 \end{bmatrix}$$

24.3 Step 1: Inizializzazione

$$Work = Available = (3, 2, 2)$$

$$Finish[i] = false \text{ per tutti i thread che detengono risorse} = [false, false, false]$$

24.4 Step 2: Trova Thread che può completare

- T_1: Request $= (1, 2, 2) \leq Work = (3, 2, 2) \rightarrow$ sì
- Simula completamento di T_1:

$$Work = Work + Allocation[T_1] = (3, 2, 2) + (2, 0, 0) = (5, 2, 2)$$

$$Finish[1] = true$$

24.5 Step 3: Continua con altri thread

- T_0: Request $= (7, 4, 3) \nleq Work = (5, 2, 2) \rightarrow$ non può completare
- T_2: Request $= (6, 0, 0) \nleq Work = (5, 2, 2) \rightarrow$ non può completare

24.6 Step 4: Fine del primo ciclo

- Nessun altro thread può completare $\rightarrow$ l'algoritmo rileva che T_0 e T_2 sono **potenzialmente in deadlock** se non cambiano le richieste future.
- T_1 ha completato e ha rilasciato le risorse subito $\rightarrow$ questo aumenta $Work$, permettendo la simulazione ottimistica.

24.7 Spiegazione Didattica

- Il rilascio immediato delle risorse detenute da un thread che può completare permette di simulare uno scenario ottimistico di avanzamento dei thread.
- Questo approccio evita di bloccare inutilmente le risorse, riducendo falsi positivi nel rilevamento dei deadlock.
- Eventuali deadlock non rilevati immediatamente saranno scoperti nelle esecuzioni successive dell'algoritmo.

24.8 Conclusione

L'algoritmo di rilevamento dei deadlock con rilascio immediato delle risorse è efficace perché:

- Minimizza il rischio di segnalare falsi deadlock
- Simula la possibilità di completamento dei thread
- Consente di individuare cicli reali di attesa tra thread che non possono completare.

25 Quando Eseguire l'Algoritmo di Rilevamento dei Deadlock

25.1 Introduzione

La decisione su quando eseguire l'algoritmo di rilevamento dei deadlock dipende da diversi fattori:

- La **probabilità stimata di deadlock** nel sistema
- Il **numero di thread** che possono essere coinvolti

25.2 Strategie di Esecuzione

- Se i deadlock sono frequenti:
  - L'algoritmo dovrebbe essere eseguito **spesso**, poiché le risorse bloccate rimangono inutilizzate e il numero di thread in attesa può aumentare rapidamente.
- Un deadlock può verificarsi quando una richiesta non può essere soddisfatta immediatamente:
  - Questa richiesta potrebbe completare una **circular wait** tra i thread.
- Strategia estrema: eseguire l'algoritmo **ogni volta che una richiesta non può essere concessa**:
  - Questo identifica immediatamente il deadlock
  - Individua anche il thread la cui richiesta ha **scatenato il deadlock**.
- Nei sistemi con più tipi di risorsa:
  - Una singola richiesta può chiudere **più cicli** contemporaneamente.
  - In tal caso, la richiesta viene segnata come **causa chiave**, anche se tutti i thread coinvolti condividono la responsabilità del deadlock.

25.3 Spiegazione Didattica

- La frequenza di esecuzione dell'algoritmo è un compromesso tra:
  - **Overhead computazionale** dovuto alla rilevazione
  - **Rischio di deadlock** e risorse bloccate
- Strategie più aggressive rilevano deadlock immediatamente, ma aumentano il carico del sistema.
- Strategie più conservative riducono l'overhead ma possono lasciare thread e risorse bloccati più a lungo.

25.4 Nota Operativa

- Nei sistemi pratici, spesso si utilizza una combinazione di:
  - Controlli periodici basati su intervalli di tempo
  - Controlli on-demand quando una richiesta non può essere soddisfatta
- Questo approccio bilancia efficienza e tempestività nella gestione dei deadlock.

26 Strategie di Recupero dai Deadlock

26.1 Introduzione

Una volta rilevato un deadlock, il sistema deve intervenire per ripristinare lo stato normale. Il recupero può essere:

- **Manuale**: l'operatore decide quali thread terminare o quali risorse liberare.
- **Automatico**: il sistema gestisce autonomamente la risoluzione del deadlock.

26.2 Metodi Principali di Recupero

Esistono due metodi fondamentali per eliminare un deadlock:

1. **Terminazione di uno o più thread coinvolti nel ciclo**
   - Interrompere un thread in deadlock rompe la **circular wait**.
   - Le risorse possedute dal thread terminato vengono liberate e rese disponibili agli altri thread.
   - La scelta del thread da terminare può basarsi su priorità, costo computazionale o stato di avanzamento.
2. **Preemption di risorse da thread bloccati**
   - Sottrarre risorse a uno o più thread bloccati e riassegnarle ad altri thread.
   - L'obiettivo è liberare risorse sufficienti per permettere ad almeno un thread di completare.
   - Successivamente, i thread che hanno subito preemption possono riprendere l'esecuzione quando ottengono nuovamente le risorse necessarie.

26.3 Obiettivo del Recupero

- L'obiettivo principale del recupero è **liberare risorse sufficienti** per consentire ai thread rimanenti di procedere.
- Idealmente, il sistema interviene minimizzando:
  - La perdita di lavoro già completato dai thread interrotti
  - L'overhead complessivo di gestione delle risorse

26.4 Spiegazione Didattica

- Il recupero dai deadlock è una strategia **reattiva**, da utilizzare solo dopo il rilevamento.
- La terminazione dei thread o la preemption delle risorse devono essere gestite con attenzione per evitare effetti collaterali significativi sul sistema.
- In pratica, i sistemi operativi combinano spesso entrambe le strategie, scegliendo in modo dinamico la soluzione più efficiente a seconda della situazione.

27 Recupero dai Deadlock tramite Terminazione dei Processi

27.1 Strategie di Terminazione

Esistono due strategie principali per terminare i processi coinvolti in un deadlock:

1. **Terminare tutti i processi in deadlock contemporaneamente**
   - Rompe immediatamente il ciclo di attesa.
   - Svantaggio: **tutto il lavoro completato dai processi terminati viene perso**, comportando uno spreco significativo di risorse e tempo.
2. **Terminare i processi uno per uno**
   - Dopo ogni terminazione, il sistema deve **rilanciare l'algoritmo di rilevamento dei deadlock** per verificare se il deadlock persiste.
   - Questo approccio riduce lo spreco di lavoro, ma richiede più tempo e calcoli ripetuti.

27.2 Criteri di Selezione dei Processi da Terminare

- La scelta del processo da terminare deve basarsi su **minimizzazione del costo complessivo** del recupero.
- Il "costo" può includere:
  - Lavoro già completato che andrebbe perso
  - Tempo perso o ritardi accumulati
  - Impatto sul sistema o su altri processi
- Spesso il costo è difficile da quantificare, quindi i sistemi operativi possono adottare euristiche o priorità predefinite.

27.3 Ripristino delle Risorse

- In tutti i casi, le risorse possedute dai processi terminati vengono **reclamate dal sistema** e rese disponibili per gli altri thread.
- Questo meccanismo è essenziale per permettere ai thread rimanenti di completare ed evitare che il deadlock persista.

27.4 Spiegazione Didattica

- La terminazione dei processi è una strategia **drastica** di recupero dai deadlock.
- È particolarmente utile quando la preemption delle risorse è complessa o inefficiente.
- La sfida principale consiste nel **bilanciare l'efficienza del recupero** con il **costo della perdita di lavoro**.
- In pratica, molti sistemi combinano terminazione selettiva e preemption per ottenere un equilibrio tra sicurezza e performance.

28 Criteri per la Scelta dei Processi da Terminare

28.1 Introduzione

Quando un deadlock viene rilevato e si opta per la **terminazione dei processi**, è fondamentale selezionare quali thread interrompere in modo strategico. La scelta influenza direttamente l'efficacia del recupero e il costo complessivo.

28.2 Principali Criteri di Selezione

1. **Priorità del processo nel sistema**
   - Processi ad alta priorità potrebbero essere protetti dalla terminazione.
   - Processi a bassa priorità sono candidati più frequenti per l'interruzione.
2. **Tempo già trascorso in esecuzione e tempo stimato al completamento**
   - Terminare un processo quasi completato comporta un **alto spreco di lavoro**.
   - Processi con poco tempo residuo sono preferibili per la terminazione se il costo complessivo deve essere minimizzato.
3. **Quantità e tipo di risorse attualmente possedute e quanto facilmente possono essere recuperate**
   - Processi che detengono poche risorse o risorse facilmente liberabili sono preferibili.
   - Processi con risorse critiche o difficili da riassegnare comportano un costo maggiore se terminati.
4. **Numero di risorse aggiuntive che il processo richiederà**
   - Processi che richiederanno molte risorse possono aumentare il rischio di deadlock in futuro.
   - Considerare questo fattore aiuta a prevenire recidive.

5. **Numero totale di processi che devono essere terminati per rompere il deadlock**
    - Minimizzare il numero di terminazioni necessarie riduce l'impatto complessivo sul sistema.
    - Una scelta ottimale cerca di risolvere il deadlock con il minor numero di interruzioni possibile.

28.3 Spiegazione Didattica

- La selezione dei processi da terminare non è mai banale: richiede bilanciamento tra **efficienza del recupero**, **minimizzazione dello spreco di lavoro** e **impatto sul sistema**.
- I sistemi operativi moderni utilizzano euristiche o combinazioni di questi criteri per prendere decisioni quasi ottimali.
- La comprensione di questi fattori aiuta a progettare algoritmi di recovery robusti e intelligenti.

29 Preemption delle Risorse nel Recupero dai Deadlock

29.1 Introduzione

Oltre alla terminazione dei processi, un'altra strategia di recupero dai deadlock è la **preemption delle risorse**. In questo approccio, il sistema **revoca risorse da uno o più processi** coinvolti nel deadlock per interrompere il ciclo di attesa.

29.2 Procedura di Preemption

1. **Selezione delle vittime**
   - Identificare i processi da cui prelevare risorse.
   - L'obiettivo è **minimizzare il costo della revoca**, riducendo l'impatto sull'esecuzione globale.
2. **Rollback**
   - I processi da cui vengono revocate le risorse devono ritornare a uno **stato precedente consistente**.
   - Lo stato coerente può includere variabili salvate, file temporanei o checkpoint.
3. **Difficoltà del rollback**
   - Se il rollback è complesso o impossibile, può essere necessario ricorrere alla **terminazione del processo**.
4. **Gestione dello starvation**
   - Evitare di selezionare sempre lo stesso processo come vittima.
   - Considerare il numero di rollback precedenti come parte dei criteri di selezione per garantire equità.

29.3 Spiegazione Didattica

- La preemption delle risorse è una strategia **meno drastica** rispetto alla terminazione dei processi.
- Permette di liberare risorse senza perdere completamente il lavoro già svolto dai processi coinvolti.
- La sfida principale consiste nel:
  - Garantire uno stato coerente dopo la revoca
  - Bilanciare il costo della preemption e del rollback
  - Evitare starvation dei processi continuamente scelti come vittime
- In pratica, sistemi avanzati combinano preemption e terminazione selettiva per ottimizzare il recupero dai deadlock.